

INFORME PROYECTO INTEGRADOR - GRUPO 19

[INTEGRANTES]

Jara Rey, Valentino.

Lopez, Hernan Lautaro.

Giménez, Lucas Nicolás

Zarate, Arturo Alan.

Aguirre, Martin

[TEMA INVESTIGADO]

-Scala

Transición

Desarrollamos esta función con el objetivo de especificar y validar las transiciones lógicas entre los distintos estados del sistema semafórico, asegurando que cumplan con las reglas establecidas en la consigna. Para ello utilizamos una estructura condicional que identifica los casos válidos y, en caso contrario, ejecuta una acción por defecto. Este diseño es adecuado porque centraliza las reglas de transición en una única función, facilita su mantenimiento y garantiza que solo se acepten cambios de estado permitidos por el sistema.

Timer

Desarrollamos **Timer** con el objetivo de determinar dentro de cuál rango se encuentran los segundos ingresados. Para ello utilizamos la operación módulo (**mod**) con 216, obteniendo el remanente de la división. Este diseño es adecuado porque permite identificar rápidamente la posición temporal dentro de un ciclo completo sin necesidad de realizar múltiples comparaciones o almacenar estados intermedios.

RegistrarAuditoria

Esta función tiene el objetivo de registrar cuándo y qué colores realizan una transición. Utilizamos la librería **local-time** para obtener la fecha y hora exactas del evento, y la función **Transicion** para validar que el cambio de estado sea correcto. Este diseño favorece la confiabilidad del registro, ya que evita almacenar transiciones inválidas.

DuraciónCiclo:

El propósito de esta función es calcular cuanto tiempo dura un ciclo y determinar si se encuentra en el rango óptimo, para ello realizamos una suma con las partes del ciclo, es decir las duraciones de rojo, verde, amarillo.

RecomendacionCiclo

La meta de esta función es hacer una recomendación para optimizar un ciclo, para lo cual utilizamos condicionales buscando si al ciclo le falta para llegar al mínimo o le sobra por haberse pasado del máximo. Este diseño es apropiado porque permite evaluar de forma clara cada posible situación y generar una recomendación específica según el resultado obtenido.

Ciclos-Por-Tiempo

El objetivo de esta función es calcular cuántos ciclos completos entran en una determinada cantidad de tiempo. Para ello dividimos el tiempo expresado en segundos por la duración de un ciclo. Esta solución es eficiente porque aprovecha directamente la relación matemática entre el tiempo total y la duración de cada ciclo, evitando cálculos innecesarios.

Distribución Porcentual

Calcula el porcentaje de aparición de los colores dentro de un rango de 3600 segundos comenzando desde el tiempo que recibe como parametro. Para ello usa regla de 3 simple, la cantidad de apariciones de un color por 100, dividido en entre la suma de las apariciones de los 3 colores. Este diseño permite representar la información de forma más intuitiva, facilitando la interpretación del comportamiento general del sistema semafórico.

VecesPeriodos

El objetivo de esta función es determinar cuántas veces aparece un estado o color dentro de un período determinado. Para ello recorre un intervalo de tiempo y contabiliza las ocurrencias correspondientes al color. Este diseño es adecuado porque centraliza el cálculo de frecuencias en una única función, permitiendo reutilizar sus resultados, como en la distribución porcentual.

Próximo-Color

El objetivo de esta función es adelantar una hora unix hasta el primer segundo del próximo color en el ciclo semafórico. Para ello resta a la hora que recibe como parámetro los segundos del ciclo actual, que se obtienen con el resto de la división entera (MOD) de la hora entre la duración del ciclo, luego suma a la hora los segundos necesarios según el color para llegar al color siguiente en el ciclo.

Complemento

Esta función recibe un color y devuelve el otro símbolo que representa el mismo color, permite tener en cuenta la versión intermitente de un color a la hora de hacer distintos cálculos o análisis.

Informe

El objetivo de esta función es generar un archivo de texto con información sobre la ejecución del sistema semafórico. Para ello utilizamos `with-open-file` para crear y escribir el informe, incorporando encabezados descriptivos y los datos obtenidos mediante `registrar-auditoria`. Este diseño es adecuado porque separa la generación del informe de la lógica principal del sistema, permitiendo almacenar los resultados de manera persistente y facilitando su consulta posterior sin necesidad de ejecutar nuevamente el programa.

Local-Time:

Utilizamos `local-time` porque proporciona un formato de fecha y hora más legible y fácil de interpretar en los registros de auditoría. Esto mejora la claridad de la información mostrada al usuario y facilita el seguimiento de las transiciones realizadas por el sistema.

Bitácora de errores

```
1
2 *** - LOAD: A file with name $file does not exist
3 Break 1 [3]>
4 #|
5 FUNCION: Transicion
6 NATURALEZA: Pura
7 ESTRATEGIA: Construccion de lista + alternativas
8 IMPACTO: No destructiva
9|#
10 (defun transicion (color-actual cambiar-a)
11 (list color-actual
12 (cond
13 ((OR (equalp cambiar-a color-actual)
14 (AND (equalp color-actual 'en-verde) (equalp cambiar-a 'rojo))
15 (AND (equalp color-actual 'en-amarillo) (equalp cambiar-a 'verde))
16 (AND (equalp color-actual 'en-rojo) (equalp cambiar-a 'amarillo))
17 ) 'accion-por-defecto)
18
19 ((equalp cambiar-a 'rojo) "cambiar-a-rojo" )
20 ((equalp cambiar-a 'amarillo) "cambiar-a-amarillo" )
21 ((equalp cambiar-a 'verde) "cambiar-a-verde")
22 (t 'accion-por-defecto)))
23 )
24
25
```

El error que cometimos en esta sección del código fue incluir una verificación explícita de los casos inválidos, en lugar de simplemente especificar los casos válidos y una salida por defecto para cualquier otra entrada. De esta forma teniendo una función más compacta.

La solución fue especificar solo los casos válidos y sólo un caso por defecto.

```
36 (defun timer(tiempo-actual)
37 (COND ((< (MOD tiempo-actual 216) 0) 'error)
38 ((<= (MOD tiempo-actual 216) 120) 'verde)
39 ((<= (MOD tiempo-actual 216) 126) 'amarillo)
40 (t 'rojo)
41 )
42 )
43
```

En este segmento el error fue a causa de una confusión con el orden de los colores del semáforo, desarrollamos la función con el orden, verde-amarillo-rojo, en lugar de utilizar el orden establecido en la consigna, rojo-amarillo-verde. Lo solucionamos corrigiendo el orden.

```
21
22 (defun veces_periodo (ini fin color)
23 (COND ((> ini fin) 0)
24 ((AND (equal (timer ini) color) (OR (not (equal (timer (+ ini 1))) (= ini fin)))
25 (+ 1 (veces_periodo (+ ini 1) fin color))
26 )
27
28 (t (veces_periodo (+ ini 1) fin color))
29 )
30 )
31
```

Esta función debía contar la cantidad de veces que aparece un color en un intervalo de tiempo, para esto cuenta el primer segundo de cada aparición del color, el error está en que

cada llamada procesa un segundo, ocasionando desbordes de la pila de memoria, incluso en intervalos menores a una hora.

```
29  #|
30  FUNCION: veces_periodo
31  NATURALEZA: pura
32  ESTRATEGIA: recursividad multiple
33  IMPACTO: no destructiva
34  |#
35
36  (defun veces_periodo (ini fin color)
37    (COND
38      ((> ini fin) 0)
39      ((equal (timer ini) color) (+ 1 (veces_periodo (proximo-color ini) fin color)))
40      (t (veces_periodo (proximo-color ini) fin color))
41    )
42  )
43
44  #|
45  CASOS DE PRUEBA
46  Comportamiento Normal:
47    (veces_periodo 1781299243 1781302843 'rojo) -> 17
48    (veces_periodo 1781299243 1781302843 'verde) -> 18
49    (veces_periodo 1781299243 1781302843 'amarillo) -> 17
50  Comportamiento Alternativo:
51    (veces_periodo 1781299243 1781302843 'no-color) -> 0
52  Caso de error:
53    (veces_periodo 0 224640 'rojo) -> Program stack overflow. RESET
54    (veces_periodo 1781299243 'no-num 'amarillo) -> >: NO-NUM is not a real number
55  |#
```

Para la solución se recorre el intervalo incrementando inicio con próximo-color, de esta forma cada llamada procesa un color del ciclo, soportando intervalos más grandes.

```
4  #| =====
5  FUNCIÓN: registrar-auditoria
6  NATURALEZA: Impura (Escribe en la salida estándar)
7  ESTRATEGIA: Simple
8  IMPACTO: No Destructiva
9  ===== |#
10 (defun registrar-auditoria (color-anterior color-nuevo)
11   (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%" (- (get-universal-time) 2208988800) color-anterior color-nuevo))
12
13
14
```

El error de esta función es que no verifica que sus parámetros se correspondan a una transición lógica de colores, o la acción por defecto, la solución es invocar la función transiciónIntermitente la cual asegura dicha lógica.

```
67  FUNCION: Ciclos-por-tiempo
68  NATURALEZA: Pura
69  ESTRATEGIA: Division tiempoTotal/ciclo
70  IMPACTO: No destructiva
71  |#
72
73  (defun ciclos-por-tiempo (minutos)
74
75    (float (/ (* minutos 60) (duracion-ciclo (* minutos 60)))))
76
```

Aquí el error es a causa de una confusión con la consigna, se entendió que los ciclos podrán ser, por ej, 3,5 ciclos, al releer la consigna nos corregimos poniendo un truncate en lugar del float, consiguiendo así la cantidad total de ciclos completos.

Presentación breve de scala

Scala (lenguaje escalable) es un lenguaje de programación creado por Martin Odersky que combina características de la programación orientada a objetos y la programación funcional. Se ejecuta sobre la Máquina virtual de Java (JVM), por lo que puede utilizar bibliotecas y herramientas del ecosistema Java. Está diseñado para desarrollar aplicaciones escalables, mantenibles y de alto rendimiento.

Áreas e industrias donde se utiliza

- Sistemas distribuidos y servicios de backend
- Big data y análisis de datos
- Procesamiento en tiempo real
- Inteligencia artificial y aprendizaje automático
- Plataformas de streaming y redes sociales
- Servicios financieros y detección de fraude

Empresas que utilizan scala

Netflix: utiliza scala para análisis de datos, sistemas distribuidos y recomendaciones personalizadas.

Linkedin: emplea scala en servicios backend, procesamiento de datos y notificaciones en tiempo real.

X: fue uno de los primeros grandes adoptantes de scala para manejar grandes volúmenes de datos y servicios de alta concurrencia.

Comparativa Lisp vs Scala

El semáforo lo estructuramos siguiendo la implementación en lisp, pero ahora respetando y siguiendo la sintaxis de scala, pero con la diferencia que ahora modelamos un semáforo como objeto singleton que sus métodos propios sean transición y timer. Para transición utilizamos un if para especificar los casos válidos e imprimirlos si se cumplen, caso contrario acción por defecto, y para Timer utilizamos un match acompañado de un if para cada caso, sabiendo así en qué rango se encuentra los segundos ingresados.

Comparando algunas funciones de orden superior de ambos lenguajes que tienen muchas similitudes entre ellos, pero scala tiene la ventaja por la notación que utiliza, en lisp se usa notación polaca la

cual no estamos acostumbrados, en cambio scala mantiene una notación más legible a simple vista.

Por último, la conclusión a la que llegamos después de comparar ambos lenguajes, es que Scala no es muy diferente de la sintaxis que ya veníamos utilizando en lisp, es más, parece que scala es como una mezcla de C y lisp, por ende la sintaxis no fue tan complicada de aprenderla, además tiene el plus de tener una notación legible o más natural para el ser humano corriente.

[BIBLIOGRAFÍA]

scala: <https://www.scalateams.com/blog/real-world-scala-company-examples>